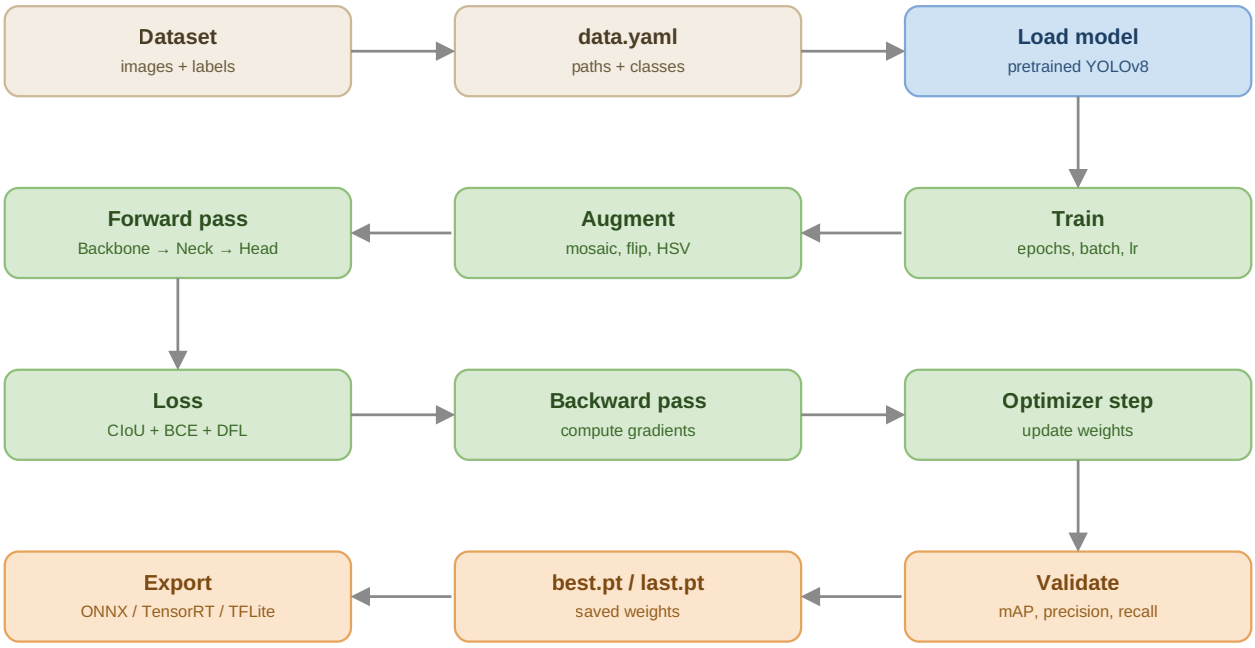
YOLOv8 — Training Pipeline Notes

Object Detection · Deep Learning Roadmap

Dataset → data.yaml → load model → train → validate → export



Train → Validate loops for every epoch, then saves weights, and can be exported

Dataset structure

- Images and labels live in separate folders: images/train, images/val, labels/train, labels/val
- Each image has a matching .txt label file with the same filename
- One line per object in the label file: class ID, then x_center, y_center, width, height
- All position/size values are normalized between 0 and 1, relative to image size

data.yaml

- A small config file that tells YOLO where the dataset lives and what the class IDs mean
- Contains the dataset path, train/val folder locations, and a list mapping class IDs to names
- Without this file, the model has no idea what class 0 or class 1 actually represents

Load pretrained model

- We start from a YOLOv8 model already pretrained on COCO (80 common object classes)
- This is transfer learning, same concept as CNN — the backbone already knows how to extract good features
- Model size choices: nano (fastest, least accurate) up to extra-large (slowest, most accurate)

> **Note:** We fine-tune the pretrained weights on our own custom classes instead of training from scratch.

Train

- Key settings: how many epochs (full dataset passes), image size, batch size, which device (GPU/CPU)
- Patience setting stops training early if validation performance stops improving, saving time

> **Note:** Internally, each training step does: augmentation (mosaic, flip, HSV shift) → forward pass (Backbone → Neck → Head) → loss calculation (CIoU for box, BCE for class, DFL for box distribution) → backward pass (gradients) → optimizer updates the weights. Validation runs automatically after every epoch.

Hyperparameter tuning

- Learning rate controls how big each weight update step is — too high causes unstable training, too low makes learning very slow
- Weight decay slightly shrinks weights every step, which helps reduce overfitting
- Augmentation strength (mosaic, hue/saturation/brightness shift, horizontal flip) can be tuned instead of using defaults

Resume training

- If training gets interrupted (crash, disconnected runtime), it can resume from the last saved checkpoint
- Resuming continues both the optimizer state and epoch count — it's a continuation, not a restart

Validate

- Runs the model only on the validation set — images it never trained on
- Reports mAP (mean Average Precision), the standard accuracy metric for object detection
- Also reports precision, recall, and per-class accuracy

> **Note:** Higher mAP means predicted boxes match ground truth boxes more closely, across all classes.

Where results get saved

- best.pt — weights from the epoch with the best validation performance (used later for inference)
- last.pt — weights from the final epoch of training
- Training curves (loss + mAP over epochs), confusion matrix, and precision-recall curve are all saved automatically as images

Export

- The trained model can be exported into different formats depending on where it will be deployed
- ONNX — cross-platform, works almost anywhere
- TensorRT — fastest option, specifically for NVIDIA GPUs
- TFLite — designed for mobile and edge devices

Inference on a new image

- The trained model can now predict on completely new, unseen images
- Each prediction returns a class ID, a confidence score, and box coordinates
- This is the same underlying process used later in real-time detection, just applied to a single static image instead of a video stream

Full pipeline recap

Stage	One line
Dataset	images + matching label .txt files
data.yaml	path to data + class names
Load model	pretrained YOLOv8, transfer learning
Train	augment → forward → loss → backward → update
Validate	mAP, precision, recall on val set
Export	ONNX / TensorRT / TFLite for deployment
Inference	predict on new unseen image

Next: YOLOv8 Real-time Detection → see separate notes file